

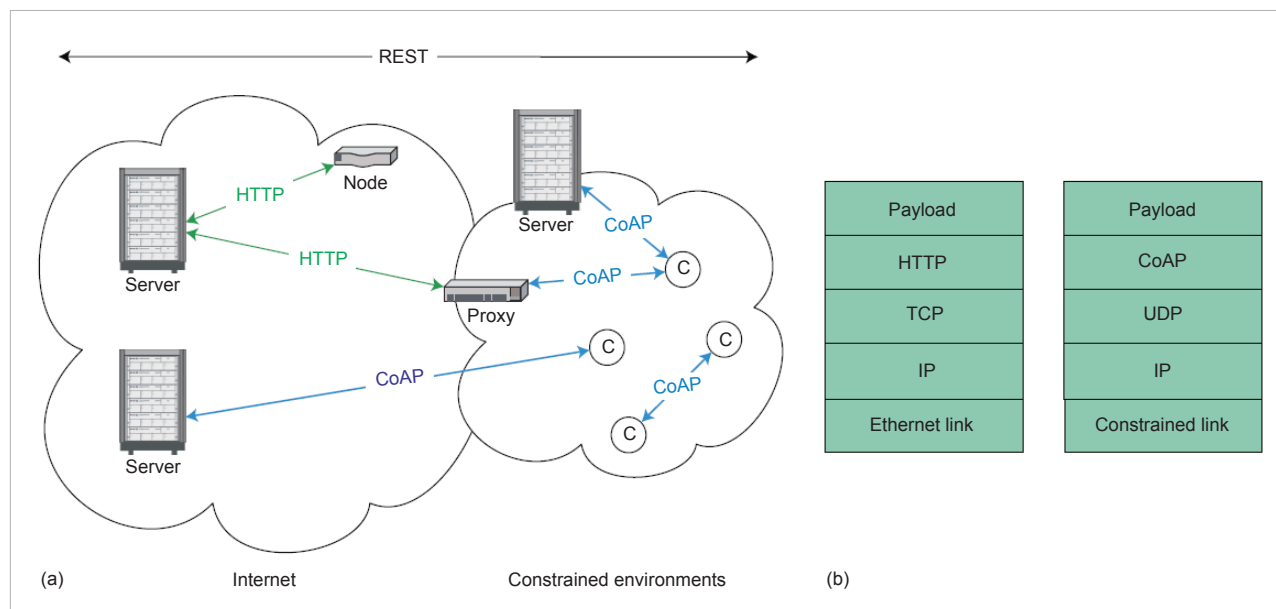


Figure 2: CoAP data transaction model (Courtesy: Devopedia)

Why do we need different protocols for IoT devices?

The rapid development of the Internet and the associated technologies that go with it has led to a surge in IoT devices. Almost everything, from your spectacles to your shoes, is now connected to the Internet. But not every device has the same processing capacity. The technology used in any wearable Apple device is far superior than, say, a basic pedometer. Add to that, Internet connectivity still remains a major issue across the globe. Even powered with the latest 5G Internet, your device is likely to suffer some sort of connectivity issues.

There are also some product-specific issues that developers come across such as the size of the device, which limits the number of chips you can fit into it. Take for instance, the latest wireless Bluetooth earbuds. The Bluetooth connectivity chips used in those devices need to be small enough to fit inside an ear and strong enough to stay connected to your phone or computer at a longer distance.

While such limitations have been a good thing, in the sense that they fuelled faster development

of revolutionary technologies, they have also given rise to new ways of communication over the Internet. Since all IoT devices cannot communicate with each other on the same capacity and capability, and have different system based requirements, we require different IoT protocols to enable smooth communication and transactions.

IoT protocols

Now that we understand why we require different IoT protocols, let us take a deeper look at each of the major ones in use today. The most prominent IoT protocols are CoAP, MQTT, XMPP, DDS, AMQP and LwM2M.

Constrained application protocol (CoAP)

When it comes to communication over the Internet, there is no protocol like the HTTP. With its underlying REST framework and simplicity, HTTP is highly reliable and easy to implement. The trouble however is that it can prove heavy for most IoT devices, as these generally require a more lightweight protocol. So can there be a protocol that has all the goodness of HTTP in

addition to being lightweight and easy to implement? That is the problem that the IETF Constrained RESTful environments working group tried to solve by creating CoAP in 2013. CoAP uses the user datagram protocol (UDP) for establishing communication between IoT devices, unlike HTTP which uses TCP.

CoAP uses a RESTful architecture and uses the HTTP GET, POST, PUT and DELETE methods. The beauty of CoAP, which distinguishes it from HTTP, is that UDP allows the IoT devices to communicate without having to establish a prior connection. This means that an IoT device using CoAP can relay multiple messages to multiple devices without having to wait for a handshake. This makes the communication faster while continuing to use very low bandwidth. To mitigate the risk of messages getting dropped due to a lack of connection, CoAP uses a confirmable/non-confirmable feature, using which messages that have been delivered can be marked confirmable. CoAP also uses datagram transport layer security (DTLS) for secure exchange of messages in the transport layer. Figure